Modules

Add a filter expression to the end of your config.include call in order to include a module (and its helper methods) conditionally. This also works with RSpec's similar config.extend and config.prepend methods, which are covered in the docs.[1]

Shared contexts

Just as with modules, add a filter expression when calling config.include_context. This will bring in your shared let constructs (among other things) into just the example groups you want.

All the techniques in this section help you selectively share behavior among your examples, based on metadata. Before we wrap up this chapter, let's take a look at one more way to use RSpec metadata: to control *how* your specs run.

Changing How Your Specs Run

RSpec allows you to change the way your specs behave using metadata. You've already used this ability several times as you've worked through this book. Here are the metadata options that affect how RSpec runs your examples:

:aggregate_failures

Changes how RSpec reacts to failure so that each example runs to completion (instead of stopping at the first failed expectation)

:pending

Indicates that you expect the example to fail; RSpec will run it and report it as *pending* if it did fail, or report it as a *failure* if it passed

:skip

Tells RSpec to skip the example entirely but still list the example in the output (unlike with filtering, which omits the example from the output)

:order

Sets the order in which RSpec runs your specs (can be the same order as they're defined, random order, or a custom order)

Both :pending and :skip take an optional explanation for *why* this spec shouldn't be run normally, which RSpec will print in the output.

As we discussed in *Testing the Invalid Case*, on page 89, RSpec is capable of running your specs in :random order, and we generally recommend you do that. The main alternative is :defined, meaning that RSpec runs your examples in the order that it sees their definitions.

1. http://rspec.info/documentation/3.6/rspec-core/RSpec/Core/Configuration.html#extend-instance_method